

Deep Learning for Multimodal Sensor Fusion

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

University of Ottawa

March 25, 2026

1. Introduction: The Paradigm Shift
2. Distance Metrics and Embeddings
3. Multimodal Embeddings
4. Deep Fusion Operators
5. Summary and Modern Tracking Stack

Why multimodal?

Multimodal learning combines complementary signals from different sensors or data sources to improve **robustness**, **coverage**, and **uncertainty awareness**, especially when one modality is noisy, missing, or ambiguous.

- **Autonomous driving and robotics**
 - Camera–LiDAR–radar fusion for **detection**, **tracking**, occupancy estimation, SLAM, localization, and navigation
 - Language + vision + action for embodied robots and human–robot interaction
- **Air/ground surveillance and defence**
 - EO/IR + radar + RF + acoustic sensing for **small-target detection**, multi-target tracking, classification, and intent inference
 - C-UAS, perimeter monitoring, and persistent wide-area sensing
- **Healthcare and biomedical AI**
 - Medical image fusion (e.g., CT/MRI/PET/ultrasound)
 - Imaging + EHR + waveform + wearable data for diagnosis, prognosis, monitoring, and triage
- **Human activity and behavior understanding**
 - Vision + IMU + audio + radar for activity recognition, pose estimation, gait analysis, and emotion/behavior monitoring
- **Remote sensing and geospatial intelligence**
 - Optical + SAR + hyperspectral + LiDAR + GIS data for land-use mapping, change detection, disaster response, and object monitoring
- **Smart cities and intelligent transportation**
 - Roadside cameras, radar, connected infrastructure, and mobile sensors for traffic analysis, pedestrian/cyclist tracking, and anomaly detection

Relevant to this course: camera–radar–LiDAR fusion for **detection**, **tracking**, **association under uncertainty**.

1.1 Traditional vs. Deep Learning Pipelines

Traditional Pipeline (Kalman + JPDA):

The classical approach relies on hand-engineered features and explicit probabilistic models:

- Extract numeric features: position $\mathbf{s} = [s_x, s_y]^T$ and velocity $\mathbf{v}$
- Compute explicit distances (Mahalanobis) between rigid state vectors
- Calculate association probabilities (JPDA) via enumeration of joint events
- Execute Kalman filter updates with fixed process and measurement models

Deep Learning Pipeline (End-to-End Fusion):

Modern approaches learn representations directly from data:

- Raw sensor data (images, LiDAR, radar) $\rightarrow$ **Latent Embeddings** $\mathbf{e} \in \mathbb{R}^d$
- Association and estimation solved **implicitly** via neural architectures
- Physical models embedded as **differentiable layers**
- Uncertainty quantified directly from learned representations

1.2 Sensor Properties and Failure Modes

Understanding complementary sensor characteristics is crucial:

每個sensor都有缺點

Sensor	Strengths	Failure Modes
Camera	Rich texture, semantics, color; dense $H \times W$ grid	Low light, fog, glare, depth ambiguity
LiDAR	Precise 3D geometry, accurate range (x, y, z, i)	Rain, dust, mirror reflections
Radar	All-weather, Doppler velocity $(r, \dot{r}, \theta)$	Low resolution, ghost targets, multipath

Key Constraint

No single sensor is sufficient. Fusion is not optional in safety-critical systems.

1.3 The Fusion Taxonomy: Where to Merge?

Early Fusion (Raw-level):

- Maximum cross-modal interaction
- Needs dense alignment
- Calibration-critical
- Example: RGB + depth concatenation

Late Fusion (Output-level):

- Maximum modularity
- Independent sensor predictions
- Less context-sharing
- Example: Combine bounding boxes

Mid Fusion (Feature-level):

- **Dominant in practice**
- Semantic features before merge
- Balance interaction/modularity
- Example: BEV fusion

BEV fusion (Bird's-Eye-View) is a multimodal perception strategy that projects each sensor's features into a common top-down grid (the BEV plane) and then fuses them there for tasks like 3D detection, mapping, occupancy, and planning.

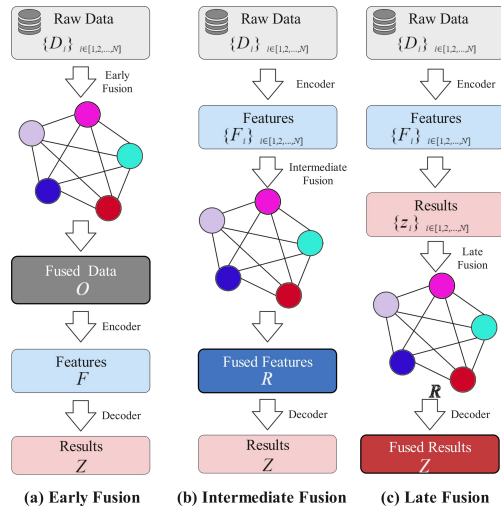
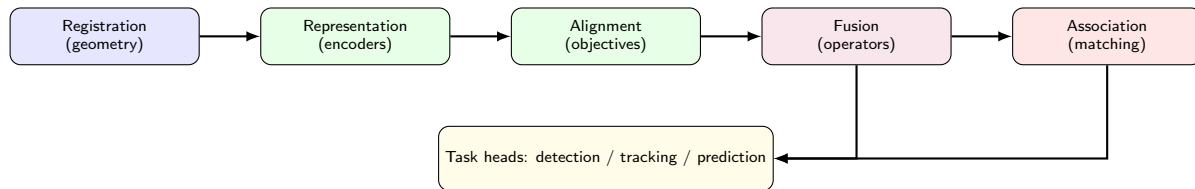


Figure: From Integrating Multi-Modal Sensors: A Review

1.4 Multimodal Tracking Block Diagram



- **Registration:** put measurements in a common *metric* frame.
- **Representation:** turn each modality into features / embeddings.
- **Alignment:** train features so matched items are comparable.
- **Fusion:** combine multi-modal features into one improved feature/state.
- **Association:** decide which detections/tracks should be fused.

Why this matters

Many deep papers blur alignment and fusion. In this lecture we keep them separate: **alignment** is an objective; **fusion** is an operator; **association** is a decision layer.

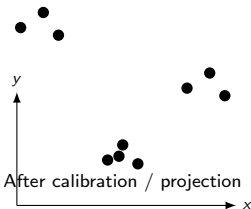
Registration, Alignment, and Association: Three Different Operations

1. Registration

Put measurements into the same physical coordinate frame

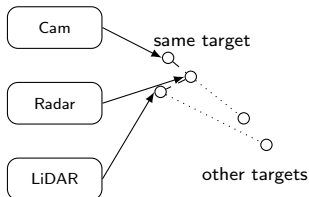
Camera

Radar



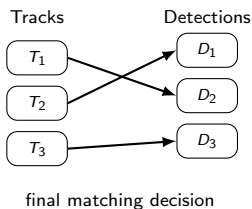
2. Alignment

Process that makes embeddings from different sources become comparable



3. Association

Use motion + geometry + embeddings to decide matches



Interpretation

Registration makes sensor data geometrically comparable. **Alignment** makes learned embeddings semantically comparable. **Association** uses these cues to decide which observations belong to the same target.

2.1 Review: The Traditional State Space

In traditional tracking (Kalman Filter), objects use rigid state vectors:

$$\mathbf{z} = [x, y, \dot{x}, \dot{y}]^T$$

Matching measurements to tracks: Mahalanobis Distance

$$D_M^2 = (\mathbf{y} - \mathbf{H}\mathbf{z})^T \mathbf{S}^{-1} (\mathbf{y} - \mathbf{H}\mathbf{z})$$

where $\mathbf{y}$ is measurement, $\mathbf{H}$ is measurement model, $\mathbf{S}$ is innovation covariance.

The Limitation

Mahalanobis distance only works on explicitly defined **physical quantities** (numbers). It cannot measure “how similar” a camera image is to a radar point cloud—they live in fundamentally different spaces.

2.2 The Deep Learning Shift: Learned Embeddings

Traditional tracking represents an object with a hand-designed physical state: $\mathbf{z} = [x, y, \dot{x}, \dot{y}]^T$

Deep learning adds a **learned representation** of the observation: $\mathbf{e} = f_{\theta}(\mathbf{X}) \in \mathbb{R}^d$ where:

- $\mathbf{X}$ is the raw input (image, radar data, point cloud, audio, etc.),
- f_{θ} is a neural network,
- d is the chosen embedding dimension (e.g., 32, 64, 128, 256).

Definition: Embedding

An embedding is a dense learned vector that places observations with similar meaning or content close together in a latent space.

Example: $\mathbf{e} = [0.12, -0.85, \dots, 0.44]^T$

This vector may encode information such as:

- object appearance,
- motion style,
- semantic category,
- cross-modal structure.

2.3 Cosine Similarity

The Metric Space Problem

In a large d -dimensional space, Euclidean distance $\|\mathbf{e}_1 - \mathbf{e}_2\|_2$ becomes numerically unstable and meaningless due to the **Curse of Dimensionality**. *Instead of measuring **distance** between points, measure **angle** between vectors.*

Cosine Similarity

For two embeddings $\mathbf{a}$ and $\mathbf{b}$:

$$\text{Sim}(\mathbf{a}, \mathbf{b}) = \cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_i a_i^2} \sqrt{\sum_i b_i^2}}$$

Interpretation - see Cosine Similarity:

- **Value = 1:** Vectors point in exact same direction (Angle = 0°). Objects identical.
- **Value = 0:** Vectors orthogonal (90°). They share no features.
- **Value = -1:** Vectors opposite (180°). Completely dissimilar.

2.4 Why Cosine Similarity over Euclidean?

Magnitude vs. Direction:

Consider a camera embedding of a car:

- In bright daylight: $\mathbf{a} = [0.8, 0.4, 0.3, \dots]$ (large magnitude)
- Same car in shadows: $\mathbf{b} = [0.4, 0.2, 0.15, \dots]$ (small magnitude, same direction!)

Euclidean distance: $\|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{(0.8 - 0.4)^2 + (0.4 - 0.2)^2 + \dots}$ is **large**

This incorrectly suggests different objects.

Cosine similarity: $\text{Sim}(\mathbf{a}, \mathbf{b}) = 1.0$ because both vectors point in the same direction!

Cosine similarity normalizes by magnitude ($\|\mathbf{a}\|\|\mathbf{b}\|$), caring only about **pattern** (angle), not scale.

Property	Dot Product	Cosine Similarity	Euclidean Distance
Definition	$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$	$\frac{\mathbf{a} \cdot \mathbf{b}}{\ \mathbf{a}\ \ \mathbf{b}\ }$	$\ \mathbf{a} - \mathbf{b}\ = \sqrt{\sum_i (a_i - b_i)^2}$
Range	$(-\infty, +\infty)$	$[-1, 1]$	$[0, +\infty)$
Sensitive to magnitude	✓ Yes	× No	✓ Yes
Measures	Direction + intensity	Direction only	Absolute distance
Higher = more similar	✓ Yes	✓ Yes	× No (lower = closer)
Equal when	Vectors are unit-normalized	—	—

Table: Comparison of dot product, cosine similarity, and Euclidean distance.

2.5 Example: Vectors in 2D Space

Imagine three vectors:

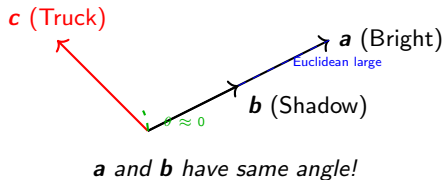
- $\mathbf{a} = [4, 2]$ (Car in bright light)
- $\mathbf{b} = [2, 1]$ (Same car in shadow)
- $\mathbf{c} = [-2, 2]$ (Truck)

Euclidean distances:

- $\|\mathbf{a} - \mathbf{b}\|_2 = 2.24$ (seems different!)
- $\|\mathbf{a} - \mathbf{c}\|_2 = 6.0$ (very different)

Cosine similarities:

- $\text{Sim}(\mathbf{a}, \mathbf{b}) = 1.0$ (identical!)
- $\text{Sim}(\mathbf{a}, \mathbf{c}) = -0.32$ (dissimilar)



$\mathbf{a}$ and $\mathbf{b}$ have same angle!

2.6 Multimodal Fusion via a Shared Embedding Space

Goal: Compare detections from different modalities such as camera and radar.

Step 1: Encode each modality into a common latent space

$$\mathbf{e}_{\text{cam}} = f_{\theta_c}(\mathbf{X}_{\text{cam}}) \in \mathbb{R}^d, \quad \mathbf{e}_{\text{radar}} = g_{\theta_r}(\mathbf{X}_{\text{radar}}) \in \mathbb{R}^d$$

where d is a chosen embedding dimension.

Step 2: Train with contrastive / metric learning

- embeddings from the **same object** are encouraged to be close
- embeddings from **different objects** are encouraged to be far apart

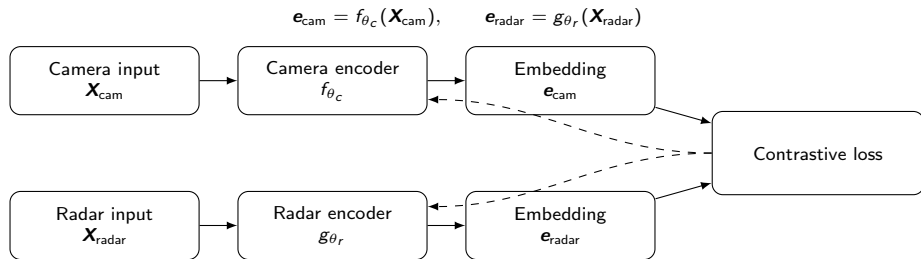
Step 3: Compare cross-modal detections Using cosine similarity,

$$\text{score}_{ij} = \frac{(\mathbf{e}_{\text{cam}}^{(i)})^\top \mathbf{e}_{\text{radar}}^{(j)}}{\|\mathbf{e}_{\text{cam}}^{(i)}\| \|\mathbf{e}_{\text{radar}}^{(j)}\|}$$

A higher score means camera detection i and radar detection j are more likely to correspond to the same object.

2.7 Learning Embeddings with Contrastive Loss: Conceptual Architecture View

Goal: learn encoder networks so that matched cross-modal inputs produce similar embeddings.



Training idea:

- same object pair $\Rightarrow$ make embeddings **close**
- different object pair $\Rightarrow$ make embeddings **far apart**. m_c is a margin that defines the maximum allowed similarity for different-object pairs. Negative pairs are penalized only when their cosine similarity exceeds this threshold.

$$\mathcal{L}_{\text{contrastive}} = \begin{cases} 1 - \cos(\mathbf{e}_{\text{cam}}, \mathbf{e}_{\text{radar}}), & \text{same object} \\ \max(0, \cos(\mathbf{e}_{\text{cam}}, \mathbf{e}_{\text{radar}}) - m_c), & \text{different object} \end{cases}$$

Key point: We do **not** learn embeddings directly. We learn the encoder parameters θ_c, θ_r , and these encoders produce embeddings that minimize the contrastive loss.

3.1 Representation Learning in Modern Tracking

Modern tracking operates in **latent spaces** rather than physical measurement spaces.

Mathematical Formulation:

Raw sensor data $\mathbf{X}_m \in \mathcal{X}$ from modality m projected via encoder $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^d$:

$$\mathbf{e}_m^{(i)} = f_\theta(\mathbf{X}_m^{(i)})$$

Properties:

- **Semantic Density:** $\mathbf{e}_m^{(i)}$ encodes geometry, appearance, and context
- **Metric Space:** Network trained such that cosine similarity $S_C(\mathbf{e}^{(i)}, \mathbf{e}^{(j)})$ is high for matched pairs and low for mismatched pairs.

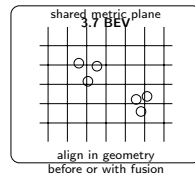
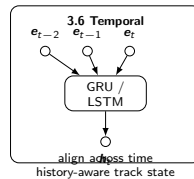
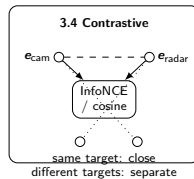
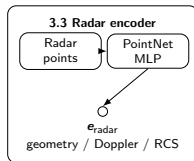
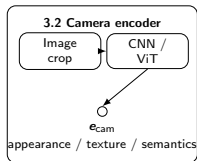
3.1a Different Roles of Learned Representations

Embeddings are not all the same. In modern tracking, learned representations can serve different purposes:

- **Modality embeddings:** per-frame camera or radar features, e.g. e_{cam} , e_{radar}
- **Alignment embeddings:** representations trained so matched pairs are close in latent space
- **Temporal embeddings:** track-history states, e.g. h_t
- **Geometric feature spaces:** BEV maps that align modalities in a common metric frame

Key distinction

An **embedding** is a representation. **Alignment** is the operation or objective that makes representations comparable across modalities, time, or space.



3.2 Visual Appearance Embeddings (Camera)

Context: Tracking-by-Detection Pipeline

- An object detector (e.g., YOLO, Faster R-CNN, DETR) first finds bounding boxes.
- For each detected object, we crop the image region:

$$\mathbf{I}_{\text{crop}} \in \mathbb{R}^{3 \times H \times W}.$$

- Each crop is processed independently to extract an *appearance embedding*.

1. CNN Feature Map

A convolutional network (e.g., ResNet) processes the crop:

$$\mathcal{M} = f_{\text{CNN}}(\mathbf{I}_{\text{crop}}) \in \mathbb{R}^{C \times H' \times W'}.$$

- Convolutions detect textures, colors, shapes.
- Deep layers capture high-level appearance (clothes, body shape, vehicle type).
- Spatial resolution shrinks (typically $H', W' \approx H/32, W/32$).

2. Global Average Pooling (GAP)

We convert the feature map into one global appearance vector:

$$\mathbf{e}_{\text{cam}} = \text{GAP}(\mathcal{M}) = \frac{1}{H'W'} \sum_{h,w} \mathcal{M}_{:,h,w} \in \mathbb{R}^{256}.$$

- GAP averages all spatial locations, producing a fixed-length descriptor.
- The embedding captures the **overall visual appearance** of the object.
- One embedding is produced **per detection**, used later for data association (Re-ID).

Summary: Detector $\rightarrow$ Crop object $\rightarrow$ CNN $\rightarrow$ GAP $\rightarrow$ Appearance embedding for matching/tracking.

3.3 Geometric & Kinematic Embeddings (LiDAR/Radar)

LiDAR/Radar frames contain **multiple objects**. We first isolate points for each detected object (3D detector (PointPillars) or clustering (DBSCAN)):

$$\mathcal{X}^{(o)} = \{\mathbf{p}_1, \dots, \mathbf{p}_{N_o}\}, \quad \mathbf{p}_i = (x, y, z, v_r, \text{RCS}).$$

PointNet-Style Embedding (per object):

1. Point-wise MLP (learn features per point)

$$\mathbf{g}_i = \text{MLP}_\theta(\mathbf{p}_i) \in \mathbb{R}^{256}$$

Intuition: Each point independently describes a tiny part of the object (surface, corner, wheel edge, micro-Doppler return). The MLP learns a feature vector for each point. No ordering required.

2. Symmetric Aggregation (max pooling)

$$\mathbf{e}_{\text{radar}}^{(o)} = \max_{i=1}^{N_o} \mathbf{g}_i \in \mathbb{R}^{256}$$

Intuition: Objects have different numbers of points (5–500). Max pooling picks the strongest signal along each feature dimension, creating a fixed-length embedding that is invariant to point order and robust to sparsity.

Result: A compact embedding capturing the object's shape, spatial layout, micro-Doppler, and RCS signature — used for multi-object tracking and association.

3.4 Multi-modal Alignment via Contrastive Learning

Problem: How ensure Radar and Camera embeddings for **same** car live in same latent space?

The loss in Slide 2.7 is a simple illustrative contrastive objective. In practice, modern multimodal systems often use batch-based variants such as InfoNCE, which we introduce next.

Solution: Information Noise Contrastive Estimation (InfoNCE Loss)(Contrastive Learning)

Given batch of B pairs $\{(\mathbf{e}_{\text{cam}}^{(i)}, \mathbf{e}_{\text{radar}}^{(i)})\}_{i=1}^B$ from same objects:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\mathbf{e}_{\text{cam}}^{(i)} \cdot \mathbf{e}_{\text{radar}}^{(i)} / \tau)}{\sum_{j=1}^B \exp(\mathbf{e}_{\text{cam}}^{(i)} \cdot \mathbf{e}_{\text{radar}}^{(j)} / \tau)}$$

Components:

- τ is learnable temperature parameter (typically 0.07)
- **Numerator:** Pulls true pair (i, i) together (maximizes similarity)
- **Denominator:** Pushes camera embedding away from all other radar embeddings

InfoNCE = Cross-Entropy Over Cosine Similarity

Goal: Align camera and radar embeddings so that matching pairs are close and non-matching pairs are far apart.

1. Cosine Similarity as a Score

If embeddings are L2-normalized:

$$s_{ij} = \frac{\mathbf{e}_{\text{cam}}^{(i)} \cdot \mathbf{e}_{\text{radar}}^{(j)}}{\tau} = \frac{\cos(\theta_{ij})}{\tau}.$$

High s_{ij} means the two embeddings point in the same direction (positive pair).

2. Cross-Entropy: Definition For a single example, the cross-entropy is defined as: $\text{CE}(p, q) = -\sum_k q(k) \log p(k)$
Where:

$q(k)$ is the target distribution, $p(k)$ is the predicted distribution (softmax output).

If the target is one-hot, i.e., the correct class is y :

$$q(y) = 1, \quad q(k) = 0 \text{ for } k \neq y,$$

then cross-entropy simplifies to:

$$\text{CE} = -\log p(y). \quad \text{because we only have 2 possibilities: probs} = \{1, 0\}$$

Cross-entropy simply encourages the correct class to have the highest softmax probability.

3. InfoNCE as Cross-Entropy

Treat matching radar embedding $j = i$ as the “correct class.” Apply softmax over cosine similarities:

$$p(j | i) = \frac{\exp(s_{ij})}{\sum_{k=1}^B \exp(s_{ik})}.$$

InfoNCE loss:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{B} \sum_{i=1}^B \log p(i | i).$$

3.5 InfoNCE Intuition (What the loss is really doing)

Per-row picture (instance discrimination).

- **Pull:** make the positive pair (i, i) very similar: maximize $e_{\text{cam}}^{(i)} \cdot e_{\text{rad}}^{(i)}$.
- **Push:** make all other pairs $(i, j \neq i)$ less similar: minimize $e_{\text{cam}}^{(i)} \cdot e_{\text{rad}}^{(j)}$.
- Intuition: the batch provides *in-batch negatives* for free; harder negatives (high cosine) are penalized most.

Training loop (practical checklist).

- 1 Sample batch of object pairs (camera & radar).
- 2 Encode and L2-normalize to get $e_{\text{cam}}, e_{\text{rad}}$.
- 3 Build similarity matrix $S_{ij} = e_{\text{cam}}^{(i)} \cdot e_{\text{rad}}^{(j)}$ and scale by $1/\tau$.
- 4 Apply cross-entropy row-wise (and optionally column-wise) against diagonal targets.
- 5 Backprop into $f_{\theta_c}, g_{\theta_r}$; repeat.

What emerges. After training, *paired* cross-modal embeddings are near cosine 1; embeddings from *different* objects/classes have much lower cosine.

3.5a After Training: How to Use the Model at Inference

You now have two encoders $f_{\theta_c}, g_{\theta_r}$ that output unit embeddings in a shared space.

A) Cross-modal matching / retrieval

- 1 Compute embeddings for a camera set and a radar set: $\mathbf{e}_{\text{cam}}, \mathbf{e}_{\text{rad}}$.
- 2 Build cosine matrix $S = \mathbf{e}_{\text{cam}} \mathbf{e}_{\text{rad}}^\top$ (values in $[-1, 1]$).
- 3 *Top-1 retrieval*: for each row (camera), pick the radar with max cosine; or *thresholding*: accept matches with $S_{ij} \geq \gamma$ (calibrate γ on validation).

B) Data association for fusion (e.g., GNN/Hungarian, JPDA)

- Turn similarity into a cost, $C_{\text{app}} = 1 - S$.
- Fuse with geometry/kinematics, e.g. $C = \alpha C_{\text{app}} + \beta C_{\text{geom}}$ (Mahalanobis).
- Solve: Hungarian (hard one-to-one) or use as likelihoods in JPDA/MHT (soft/exclusive).

C) Pretraining for BEV fusion

- Use InfoNCE-aligned encoders to initialize a downstream BEV fusion model; fine-tune with detection/segmentation heads.

3.5b InfoNCE: Worked Numerical Example ($B = 3, \tau = 1$)

Assume embeddings are L_2 -normalized, so

$$\mathbf{e}_{\text{cam}}^{(i)\top} \mathbf{e}_{\text{radar}}^{(j)} = \cos(\mathbf{e}_{\text{cam}}^{(i)}, \mathbf{e}_{\text{radar}}^{(j)}).$$

With $\tau = 1$, the logits are

$$S_{ij} = \frac{\mathbf{e}_{\text{cam}}^{(i)\top} \mathbf{e}_{\text{radar}}^{(j)}}{\tau} = \cos(\mathbf{e}_{\text{cam}}^{(i)}, \mathbf{e}_{\text{radar}}^{(j)}),$$

so each entry must lie in $[-1, 1]$.

For this toy batch,

$$\mathbf{S} = \begin{pmatrix} \mathbf{0.8} & -0.2 & 0.1 \\ 0.1 & \mathbf{0.7} & -0.3 \\ -0.1 & 0.2 & \mathbf{0.9} \end{pmatrix}$$

Diagonal entries are the true camera–radar matches.

For each camera embedding $\mathbf{e}_{\text{cam}}^{(i)}$, the row softmax turns similarities to all radar embeddings into a probability distribution:

$$p_{ij} = \frac{\exp(A_{ij})}{\sum_{k=1}^B \exp(A_{ik})}.$$

The loss for row i is

$$\mathcal{L}_i = -\log p_{ii}.$$

Interpretation: Row i asks: “Given camera sample i , which radar sample is its correct match?” The correct answer should be the diagonal entry $j = i$. Thus, InfoNCE is a **cross-entropy classification loss over the batch**.

Row 1:

$$p_{11} = \frac{e^{0.8}}{e^{0.8} + e^{-0.2} + e^{0.1}} = \frac{2.23}{2.23 + 0.82 + 1.11} = \frac{2.23}{4.15} \approx 0.54$$

$$\mathcal{L}_1 = -\log(0.54) \approx 0.62$$

Full loss:

$$\begin{aligned} \mathcal{L}_{\text{InfoNCE}} &= -\frac{1}{3} \left[\log \frac{e^{0.8}}{4.15} + \log \frac{e^{0.7}}{3.86} + \log \frac{e^{0.9}}{4.59} \right] \\ &\approx \frac{0.62+0.65+0.62}{3} \approx 0.63 \end{aligned}$$

What happens next? Backpropagation updates both encoders to **increase diagonal similarities** and **decrease off-diagonal similarities**, so matched pairs move closer and mismatched pairs move farther apart.

3.6 Building Temporal Embeddings from Track History

Goal: convert a sequence of per-frame detections into a **track memory** that summarizes motion, appearance, and reliability over time.

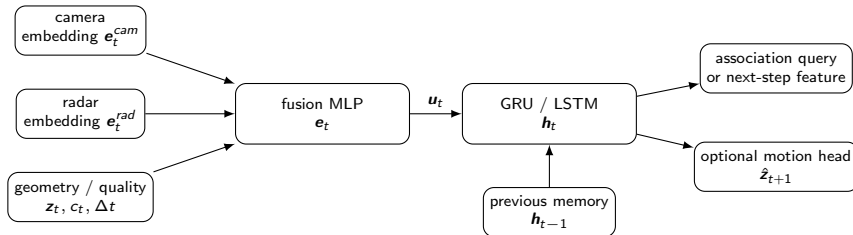
$$\mathbf{e}_t = \phi_{\text{fuse}}(\mathbf{e}_t^{\text{cam}}, \mathbf{e}_t^{\text{radar}}, \mathbf{z}_t, c_t)$$

where:

- $\mathbf{e}_t^{\text{cam}}, \mathbf{e}_t^{\text{radar}}$ are per-frame modality embeddings,
- $\mathbf{z}_t$ is current geometry (e.g. position/velocity box state),
- c_t is confidence / quality metadata.

Temporal update:

$$\mathbf{u}_t = [\mathbf{e}_t; \Delta \mathbf{z}_t; \Delta t; c_t], \quad \mathbf{h}_t = \text{GRU}(\mathbf{u}_t, \mathbf{h}_{t-1})$$



Interpretation:

- $\mathbf{e}_t$ = what the target looks like *now*
- $\mathbf{h}_t$ = what the tracker remembers about this target *over time*

3.7a Bird's-Eye View (BEV): A Common Map for Fusion

Core problem: camera, LiDAR, and radar all observe the same world, but each sensor represents it differently. To fuse them, we need a **common reference space**.

One-sentence definition

Bird's-Eye View (BEV) is a shared 2D top-down grid where every sensor's data is projected, so all modalities can be fused in the same space.

Intuition: Think of BEV as a **Google Maps view** of the scene. No matter whether information comes from a camera, LiDAR, or radar, everything ends up as features on the same top-down map.

Connection to classical fusion:

Classical Fusion	Deep Fusion (BEV)
Common ECEF / Cartesian frame	Common BEV grid
Global coordinate system tied to the Earth	Local around the ego vehicle
Fuse only after registration	Fuse only after projection to BEV

Why not use the camera image directly?

- **Perspective distortion:** distant objects look small, nearby ones large
- **Occlusion:** objects can hide behind one another

Key message BEV is to deep multimodal fusion what ECEF is to classical sensor fusion: a **universal meeting ground**.

ECEF stands for Earth-Centered, Earth-Fixed. It's a Cartesian coordinate system used in navigation and sensor fusion to represent positions, velocities, and orientations relative to the Earth.

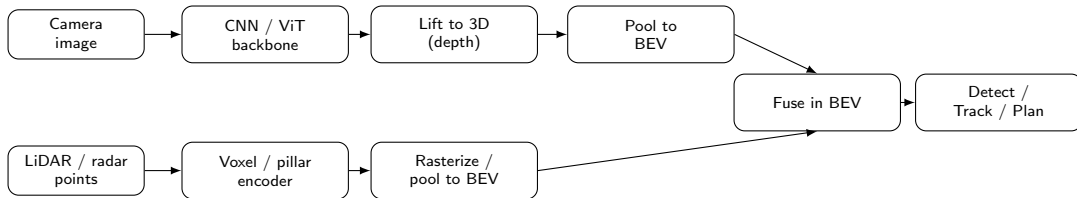
3.7b How BEV Is Built and Why It Helps

LiDAR / radar branch (easy):

- Input: LiDAR already gives (x, y, z) 3D points in metric space and radar gives radar detections
- For LiDAR, simply drop the height (z) or aggregate it, and you have a top-down map. This is called voxelization (3D cubes) or pillars (vertical columns collapsed to 2D).
- Output: top-down BEV feature map

Camera branch (hard):

- Input: 2D image features from a CNN / ViT
- **Lift**: estimate depth and move image features into 3D space
- **Project / Pool**: place those 3D features onto the BEV grid
- Output: camera BEV feature map



$$F_{\text{BEV}}^{\text{cam}} = \text{Pool}_{\text{BEV}}(\text{Lift}(F_{\text{img}}, D)), \quad F_{\text{BEV}}^{\text{lid}} = \text{Rasterize}(P)$$

Why BEV makes fusion easier:

- camera contributes **semantics**
- LiDAR / radar contributes **geometry**
- both are now aligned in the same top-down metric grid

Why BEV Is Unavoidable

BEV is the **de facto standard output space** for any perception system that feeds into a planner.

The planner always works in the **ground plane** — a car drives on a 2D surface — so the *entire* perception stack is designed to produce BEV-ready outputs.

Autonomous Driving Tasks

- **3D Object Detection** — boxes as $(x, y, z, l, w, h, \theta)$
- **Multi-Object Tracking** — associate detections in metric space
- **HD Map Construction** — lanes, boundaries as ground-plane polylines
- **Motion Prediction** — future (x, y) trajectories
- **Occupancy Grids** — which BEV voxels are occupied?
- **End-to-End Planning** — UniAD, VAD run fully in BEV

Beyond Autonomous Driving

- **Robotics** — ROS occupancy grids *are* BEV costmaps
- **Warehouse robots** — Amazon Kiva, AMR path planning
- **Drones / UAVs** — their native view *is* BEV
- **Satellite imagery** — land use, change detection
- **Sports analytics** — player tracking on pitch/court
- **Retail analytics** — overhead customer flow maps

4.1 Fusion Operator: Concatenation + MLP

Definition

$$\mathbf{e}^{fuse} = \text{MLP}([\mathbf{e}^{cam}; \mathbf{e}^{rad}; \mathbf{e}^{lid}])$$

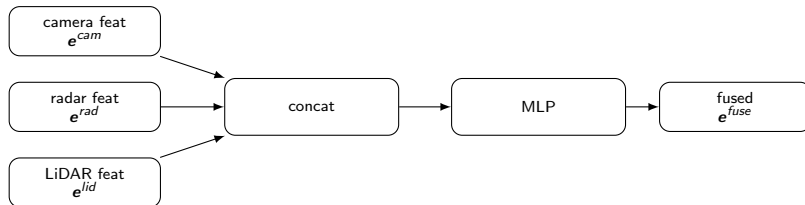
Pros

- Simple, fast, widely used baseline
- Learns non-linear mixing across modalities

Cons

- Requires all modalities present (unless you mask)
- Can overfit to spurious correlations
- No explicit notion of reliability per modality

Best for: mid-level feature fusion when modalities are synchronized.



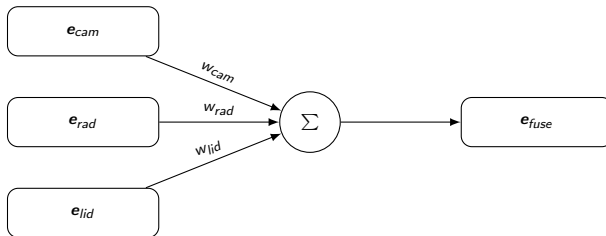
4.2 Fusion Operator: Weighted Sum / Confidence-Weighted Fusion

Idea: compute a reliability weight per modality, then fuse by a weighted sum.

$$\mathbf{e}_{fuse} = \sum_{m=1}^M w_m \mathbf{e}_m, \quad \sum_m w_m = 1, \quad w_m \geq 0$$

Where do weights come from?

- **Heuristic confidence:** detector score, SNR, number of points
- **Learned confidence:** $w = \text{softmax}(W[\mathbf{e}^1; \dots; \mathbf{e}^M])$
- **Uncertainty-aware:** $w_m \propto 1/\sigma_m^2$ (see uncertainty slide)



4.3 Fusion Operator: Gated Fusion

Idea: A gated fusion first uses a small learned network to decide how much of each feature should pass through, then fuses them.

Two-modality example:

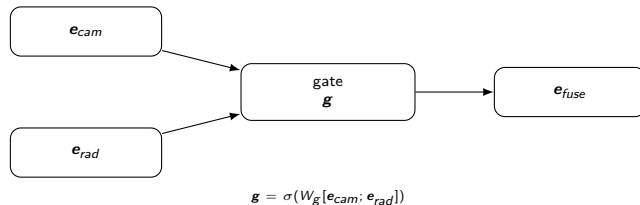
$$\mathbf{g} = \sigma(W_g[\mathbf{e}_{cam}; \mathbf{e}_{rad}])$$
$$\mathbf{e}_{fuse} = \mathbf{g} \odot \mathbf{e}_{cam} + (1 - \mathbf{g}) \odot \mathbf{e}_{rad}$$

Interpretation

- if $\mathbf{g}$ is near 1 $\Rightarrow$ rely on camera features
- if $\mathbf{g}$ is near 0 $\Rightarrow$ rely on radar features
- can be **global** (one scalar) or **fine-grained** (vector gate)

Pros: handles missing / degraded modalities better than plain concatenation.

Cons: gate can collapse (always picks one modality) without regularization.



4.4 Fusion Operator: Mixture-of-Experts (MoE) and Product-of-Experts (PoE)

Mixture-of-Experts (choose who to trust)

$$\mathbf{e}_{fuse} = \sum_{k=1}^K \alpha_k(\text{context}) \text{Expert}_k(\mathbf{e}_1, \dots, \mathbf{e}_M)$$

- gating network outputs α_k (soft selection)
- experts can specialize: weather, speed regime, sensor configuration

Product-of-Experts (combine independent evidence) If each modality provides a probabilistic estimate:

$$p(\mathbf{z} \mid \mathbf{y}^{(1)}, \mathbf{y}^{(2)}) \propto p(\mathbf{z} \mid \mathbf{y}^{(1)}) p(\mathbf{z} \mid \mathbf{y}^{(2)})$$

For Gaussians, precisions add: $\Lambda = \Lambda_1 + \Lambda_2$.

When to mention PoE in this course

PoE is a clean bridge to **classical Bayesian fusion**: multiply likelihoods / add information matrices.

Pros

- MoE: handles heterogeneous regimes, partial sensors
- PoE: principled when modalities are conditionally independent

Cons

- MoE: training instability, expert collapse
- PoE: overconfident if independence assumption is violated

4.5 Uncertainty in Multimodal Fusion (What to Model, Where to Use It)

What uncertainty shows up in fusion?

- **Aleatoric:** sensor noise, weather, low light, multipath
- **Epistemic:** model uncertainty (OOD scenes, rare objects)
- **Calibration / timing:** imperfect extrinsics, latency, sync drift

How to use it (practically):

- **Reliability-weighted fusion:** $w_m \propto 1/\sigma_m^2$
- **Gating / fallback:** if uncertainty high, rely on robust modality or classical model
- **Conflict detection:** disagreeing modalities $\Rightarrow$ raise uncertainty

Common modeling choices

- **Heteroscedastic outputs:** predict $\sigma^2(\mathbf{x})$ and train with NLL
- **Deep ensembles:** variance across models
- **Evidential methods:** output distribution parameters with uncertainty

Graduate-level takeaway

Fusion is not only “combine features”; it is also **manage reliability**. A strong fusion module should degrade gracefully when a modality fails.

5.1 Key Conceptual Bridges

Understanding connection between classical and deep methods:

Classical	Deep
Mahalanobis distance in physical space	Cosine similarity in embedding space
JPDA β_{ij} matrix via marginalization	Attention weights A_{ij} via learned features
ECEF or local Cartesian frame	Bird's-Eye View (BEV)
Fixed gating thresholds (χ^2)	Learned attention masks and heteroscedastic R_t
Hard-coded physics (F , Q)	Embedded differentiable layers (DKF)

5.2 When to Use Each Approach

Classical Methods (Kalman + JPDA) Best When:

- Small amounts of data
- Well-understood physics dominates
- Interpretability critical
- Real-time constraints with limited compute
- Safety certification required

Deep Methods Best When:

- Large datasets available
- Complex appearance/semantic features
- Multi-modal fusion
- Calibration imperfections expected
- Domain adaptation required

Hybrid Methods (DKF) Best When:

- Need physics guarantees AND adaptability
- Limited data but want priors
- Interpretable uncertainty quantification
- Gradual transition classical $\rightarrow$ deep

Recommendation

Most safety-critical applications benefit from hybrid approaches combining physics-based priors with learned components.

1. **Physics-Informed Neural Networks:**

- Embedding PDEs and conservation laws in loss functions
- Differentiable simulation for robotics
- Neural ODEs with guaranteed stability

2. **Test-Time Adaptation:**

- Continuous learning during deployment
- Domain adaptation without labeled data
- Meta-learning for few-shot adaptation

3. **Uncertainty-Aware Planning:**

- Using epistemic uncertainty for active perception
- Risk-sensitive decision making
- Conformal prediction for safety guarantees

4. **Multimodal Foundation Models:**

- Large-scale pre-training on diverse sensor data
- Zero-shot transfer to new sensor configurations
- Self-supervised learning from raw sensor streams

5.4 Practical Implementation Considerations

Training Strategies:

- **Curriculum Learning:** Start single modalities, gradually introduce fusion
- **Auxiliary Losses:** Per-modality detection, intermediate feature matching, physics consistency
- **Data Augmentation:** Sensor dropout, calibration perturbation, weather simulation

Architecture Choices:

- **Backbone:** Camera (ResNet-50, ViT), LiDAR (PointPillars, CenterPoint), Radar (PointNet)
- **Fusion:** Early (tight alignment), Mid/BEV (versatile, recommended), Late (modularity)
- **Temporal:** GRU/LSTM (simple), TCN (parallel), Transformer (SOTA but expensive)

Deployment:

- **Latency:** Autonomous driving $< 100\text{ms}$, Surveillance $< 1\text{s}$
- **Hardware:** Edge (quantization, pruning), Cloud (ensembles), Embedded (classical still dominant)
- **Failure:** Graceful degradation, uncertainty thresholding, fallback to classical

The field has moved from **hard-coded physical filters** to **differentiable, end-to-end learning pipelines** where:

- Physics acts as **structural prior** (inductive bias)
- Uncertainty quantified directly from latent representations
- Fusion happens in learned feature spaces (BEV, embeddings)
- Association is soft and learned (attention)

The Key Insight:

Deep learning and classical methods are not antagonistic, but **complementary**.

Most powerful systems combine:

- **Sample efficiency** and **guarantees** of physics-based methods
- **Flexibility** and **semantic understanding** of deep learning

Embeddings and Contrastive Learning:

- Chen et al. (2020). "SimCLR: A Simple Framework for Contrastive Learning"
- Radford et al. (2021). "CLIP: Learning Transferable Visual Models"

Multimodal Fusion Architectures:

- Vora et al. (2020). "PointPainting: Sequential Fusion for 3D Object Detection"
- Phillion & Fidler (2020). "Lift, Splat, Shoot"
- Bai et al. (2022). "TransFusion: Robust LiDAR-Camera Fusion"

Attention and Transformers:

- Vaswani et al. (2017). "Attention Is All You Need"
- Dosovitskiy et al. (2021). "Vision Transformer (ViT)"

Physics-Informed DL:

- Kloss et al. (2021). "How to Train Your Differentiable Filter"
- Karniadakis et al. (2021). "Physics-Informed Neural Networks"

Thank you for your attention!

Questions and Discussion

This document was developed and converted into LaTeX slides and formatted with assistance from ChatGPT (OpenAI) and CoPilot (Microsoft). The instructor modified the source text and verified the final structure and wording.